University of Moratuwa

Department of Electronic and Telecommunication Engineering

EN3150 – Assignment 03

Resource-Constrained CNNs for EuroSAT Classification

Group name:	Kernel
Course:	EN3150 – Pattern Recognition
Submission date:	19/11/2026

Repository: https://github.com/rus1ru/EN3150_A03

Group Members

Group name: Kernel

Member	Name	Index number
Member 1	Rasanjana W.P.G.R.A.	230536E
Member 2	Karunarathna G.K.T	230322U
Member 3	Ubeysekara V.T.T.	230650X
Member 4	Imaduwage O.N.H.	230258D

Contents

Group Members **i**

1 Introduction **1**

2 Dataset and protocol **1**

3 Architectures and parameter accounting **1**
3.1 Model A: standard convolution 1
3.2 Model B: custom depthwise network 2
3.3 Pretrained references 3

4 Training settings **3**

5 Results **3**

6 Discussion **4**
6.1 Convergence and classification errors 4
6.2 Limitations 5

7 Conclusion **5**

A Experiment records **10**

Abstract

We compare two custom convolutional networks on EuroSAT RGB under a matched, single-seed training protocol. Both models use the same split, normalization, augmentation, batch size, 40-epoch budget and optimizer settings. Model A achieves 95.48% test accuracy with Adam and 95.95% with momentum SGD. Model B achieves 96.69% and 97.14%, respectively, with 96,538 parameters, below the 100,000-parameter limit. Separate pretrained MobileNetV2 and EfficientNet-B0 reference experiments achieve 97.65% and 97.85%. Model B offers lower parameter storage and counted arithmetic than Model A, but the comparison concerns complete architectures, not convolution type alone. Single-seed results and prior test-set reuse limit generalization claims.

1 Introduction

Deploying an image classifier on limited hardware requires balancing accuracy, weight storage and computation. Standard convolutions jointly learn spatial and channel features, whereas depthwise-separable convolutions split these operations to reduce arithmetic and parameter cost [**?**]. Pretrained lightweight networks provide an alternative: they reuse features learned on a larger dataset, but may still exceed a small custom network's storage budget.

This study compares a standard CNN (Model A), a custom depthwise CNN (Model B), and fine-tuned MobileNetV2 and EfficientNet-B0. We examine three questions: how the custom models respond to different optimizers, whether Model B meets the 100,000-parameter constraint without a large accuracy loss, and how its accuracy and theoretical cost compare with the pretrained references.

2 Dataset and protocol

EuroSAT provides satellite imagery for land-use and land-cover classification [**?**]. We use the official RGB release from Zenodo record 7711810: 27,000 RGB images, ten classes, native 64×64 resolution. The frozen class-stratified split contains 18,900 training, 4,050 validation and 4,050 test images, using seed 42. The split manifests are shared across models; verification found no overlapping image paths between partitions.

Classes are AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River and SeaLake, in that order. Augmentation and normalization fitting use training data only. Metrics use the complete 4,050-image test split; confusion-matrix rows represent true classes and columns predicted classes. We report accuracy and macro precision/recall with zero-denominator scores set to zero. Checkpoints are chosen using validation loss for A/B/MobileNetV2, and validation accuracy for EfficientNet-B0.

3 Architectures and parameter accounting

3.1 Model A: standard convolution

Model A contains four 3×3 convolutions. Every convolution is followed by ReLU and 2×2 max pooling. Channel/spatial sizes are $3 \times 64 \times 64 \rightarrow 32 \times 32 \times 32 \rightarrow 64 \times 16 \times 16 \rightarrow 96 \times 8 \times 8 \rightarrow$

$112 \times 4 \times 4$. Global average pooling produces 112 features, followed by a $112 \to 192 \to 10$ classifier with ReLU between linear layers. Standard convolutions/max pooling are established CNN operations [**?**]; global pooling avoids a large flattened head [**?**].

Layer	Calculation (including bias)	Parameters
Conv $3 \to 32$	$32(9 \cdot 3 + 1)$	896
Conv $32 \to 64$	$64(9 \cdot 32 + 1)$	18,496
Conv $64 \to 96$	$96(9 \cdot 64 + 1)$	55,392
Conv $96 \to 112$	$112(9 \cdot 96 + 1)$	96,880
Linear $112 \to 192$	$112 \cdot 192 + 192$	21,696
Linear $192 \to 10$	$192 \cdot 10 + 10$	1,930
Total		195,290

ReLU is a max-with-zero operation without exponentials. Pooling reduces activation sizes. Parameter storage is not peak runtime memory.

3.2 Model B: custom depthwise network

Model B combines depthwise/pointwise convolution [**?**], inverted residuals and linear projections [**?**], and squeeze-and-excitation channel gates [**?**]. It is a custom configuration trained from scratch, not a pretrained MobileNet or a claimed new operation. An RGB pointwise lift creates 24 channels before the first strided depthwise filter. Seven expanded bottlenecks use expansion ratio two, widths 24, 32, 32, 48, 48, 80, 80, and four residual additions. Two SE gates are used. A pointwise head produces 192 channels, followed by global average pooling, dropout 0.1 and ten logits. Batch normalization and ReLU6 are used; hard-sigmoid gates avoid an exponential at inference. Framework profiling gives a total of **96,538 trainable parameters**, including normalization and attention parameters, below the 100,000 cap. A depthwise/pointwise pair without biases has $9C_{in} + C_{in}C_{out}$ weights; normalization, linear layers and SE weights must also be included.

Table 1: Model B stage shapes and parameter counts, including normalization and attention. Shapes exclude the batch dimension.

Stage	Output ($C \times H \times W$)	Parameters
Pointwise RGB lift	$24 \times 64 \times 64$	120
Depthwise stem	$24 \times 32 \times 32$	264
Inverted residual 1	$24 \times 32 \times 32$	2,976
Inverted residual 2	$32 \times 32 \times 32$	3,376
Inverted residual 3	$32 \times 32 \times 32$	4,992
Inverted residual 4	$48 \times 16 \times 16$	6,048
Inverted residual 5	$48 \times 16 \times 16$	12,972
Inverted residual 6	$80 \times 8 \times 8$	13,696
Inverted residual 7	$80 \times 8 \times 8$	34,420
Head, pooling and classifier	10	17,674
Total		96,538

3.3 Pretrained references

MobileNetV2 [**?**] uses ImageNet weights, three classifier-head warmup epochs then full fine-tuning, for 20 total epochs. EfficientNet-B0 [**?**] replaces its classifier with ten outputs and fine-tunes all weights for ten epochs. Both retain 64×64 inputs and apply ImageNet normalization and training-only horizontal/vertical flips. These are reference recipes, not matched-budget architecture ablations.

4 Training settings

The controlled custom-model study uses seed 42, the frozen shared split and the RTX 4050 Laptop GPU with PyTorch 2.4.1+cu121. Both architectures are unchanged. Each is trained independently for 40 epochs per optimizer, with batch size 64, plain cross-entropy and FP32 arithmetic (AMP and TF32 disabled). Both use the same training-pixel mean/std normalization and independent horizontal and vertical flips with probability 0.5. Validation and test apply normalization only.

Both models use Adam at learning rate 0.001, plain SGD at 0.01, and SGD with momentum 0.9 at 0.01. Weight decay is zero; there is no scheduler, label smoothing or early stopping. Adam provides an adaptive-step baseline, while plain and momentum SGD expose the effect of gradient accumulation under fixed settings. These learning rates were fixed before this study, not selected by a learning-rate search. Equal settings do not guarantee each architecture's optimal performance.

Within each architecture, the three optimizers begin from identical initialization. Training shuffle and augmentation streams are reset for each recipe. All six runs finish before any test evaluation. Checkpoints are selected by minimum validation cross-entropy, then evaluated on the full test split. CuDNN deterministic settings reduce variation but do not guarantee bitwise GPU reproducibility.

The pretrained references are separate, unmatched experiments. MobileNetV2 uses 20 epochs, batch size 32, Adam with head learning rate 0.001 then 0.0001, and CPU execution. EfficientNet-B0 uses ten epochs, batch size 32, Adam at 0.0001, and CUDA execution. They are not part of the controlled A/B training comparison.

5 Results

One MAC denotes one multiply-accumulate (approximately two FLOPs). Counts cover Conv2d and Linear operations, including SE convolutions; they exclude bias additions, activations, pooling, normalization, residual additions, SE elementwise multiplication, framework overhead and memory traffic. They are not deployment benchmarks. Actual checkpoint size also includes buffers/metadata and is stored separately in run artifacts. In decimal MB (10^6 bytes), FP32 parameter storage is 8.95 MB for MobileNetV2 and 16.08 MB for EfficientNet-B0.

Model	Optimizer	Accuracy (%)	Precision (%)	Recall (%)	s/epoch
Model A	Adam	95.48	95.33	95.39	9.02
Model A	SGD	83.51	82.98	82.84	8.23
Model A	SGD + momentum	95.95	95.93	95.89	9.82
Model B	Adam	96.69	96.70	96.55	14.90
Model B	SGD	93.06	92.86	92.88	14.12
Model B	SGD + momentum	97.14	97.07	97.04	14.10
MobileNetV2	Adam (two-stage)	97.65	97.64	97.46	—
EfficientNet-B0	Adam	97.85	97.86	97.76	16.79

Table 2: Recorded test metrics. Precision and recall are macro averages. Timing is mean training-plus-validation time per epoch, not inference latency. A dash denotes timing not reported in this comparison.

Model	Parameters	FP32 weights (KiB)	Conv/linear MACs
Model A	195,290	762.85	42,785,664
Model B	96,538	377.10	21,358,976
MobileNetV2	2,236,682	8737.04	24,461,312
EfficientNet-B0	4,020,358	15704.52	31,979,392

Table 3: Parameter and operation accounting at 64×64, batch one. FP32 storage is parameters times four bytes divided by 1,024; it excludes activations and optimizer state.

6 Discussion

Model B exceeds Model A by 1.21 percentage points with Adam, 9.56 points with plain SGD, and 1.19 points with momentum SGD. Momentum SGD improves on plain SGD by 12.44 points for Model A and 4.07 points for Model B. Momentum can accumulate consistent gradient directions, but these results apply to the fixed learning rates and one seed; they do not establish universal optimizer superiority.

Model B has approximately 50.6% fewer parameters and 50.1% fewer counted MACs than Model A. Its accuracy advantage persists under the matched training protocol. However, it also differs through batch normalization, residual additions, attention, dropout, activation functions and initialization. The experiment therefore compares complete architectures, not an isolated standard-to-depthwise replacement.

The separately trained MobileNetV2 and EfficientNet-B0 references score 97.65% and 97.85%. Model B with momentum SGD records 97.14%, while using substantially fewer parameters. These reference scores provide context, not a matched-budget comparison or statistical ranking.

6.1 Convergence and classification errors

The largest directed off-diagonal counts in the displayed custom-model matrices are Model A momentum SGD: HerbaceousVegetation to PermanentCrop (17), PermanentCrop to HerbaceousVegetation (12), River to AnnualCrop (10). Model B Adam: PermanentCrop to HerbaceousVegetation (19), Industrial to Residential (11), Industrial to Highway (11). These identify observed errors; inspecting the images would be necessary to establish their visual causes.

Figures 1 and 2 show loss and accuracy histories for the controlled recipes. Both now use the same plain cross-entropy definition, although training augmentation and Model B's training-

only dropout still affect training/validation differences. Figure 3 displays Model A momentum SGD and Model B Adam, together with the pretrained references. All six custom-model results are reported in the table; the displayed matrices are illustrative, not a new test-based selection rule.

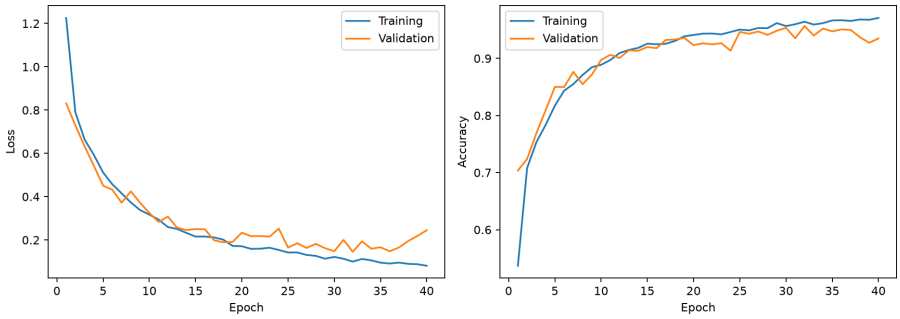
6.2 Limitations

The experiments use one seed and an image-level split, not a geographic holdout; spatially related satellite tiles can limit generalization. Earlier test scores were visible before architecture and epoch-budget changes. This adaptive test reuse means the results are exploratory rather than an untouched final assessment. An independent holdout and repeated seeds would be needed for stronger comparisons.

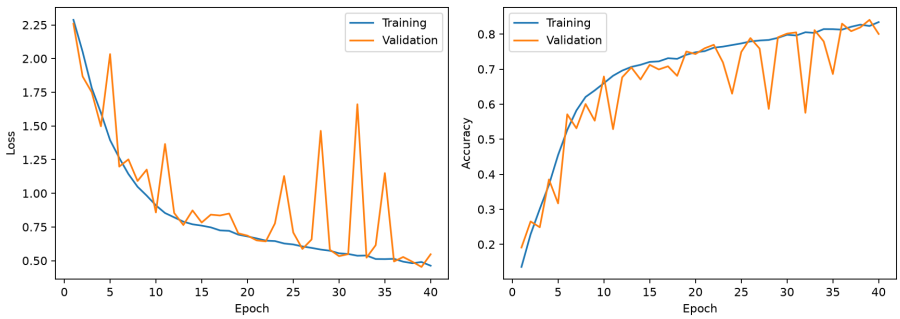
The custom models now share augmentation, normalization, optimizer settings, batch size, loss and epoch budget. Nevertheless, one fixed learning rate per optimizer and one seed cannot characterize optimal performance or uncertainty. Pretraining and different budgets distinguish the reference networks. Timing is also recipe- and hardware-dependent: MobileNetV2 ran on CPU, whereas the new custom runs and EfficientNet-B0 ran on CUDA. Parameter counts and MACs do not establish inference latency, energy use or peak runtime memory.

7 Conclusion

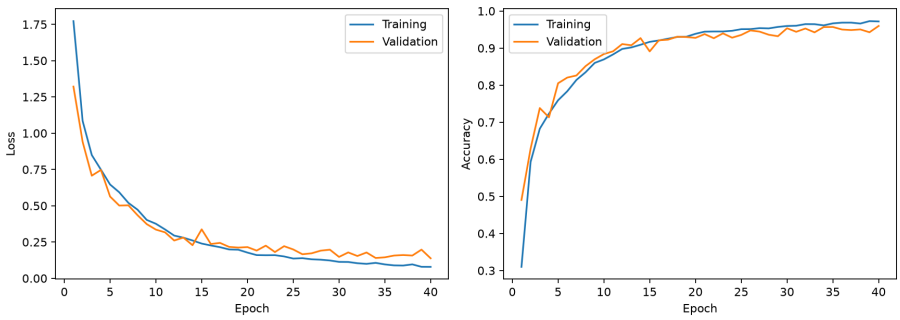
Under the recorded recipes, the custom sub-100k Model B attains accuracy close to the pretrained references while using substantially fewer parameters. Model B scores higher than Model A for each optimizer under the matched training protocol; repeat-seed studies are needed to assess the stability of these differences. Resource claims are restricted to parameter storage and explicitly counted operations; deployment speed and peak runtime memory remain unmeasured.



(a) Adam

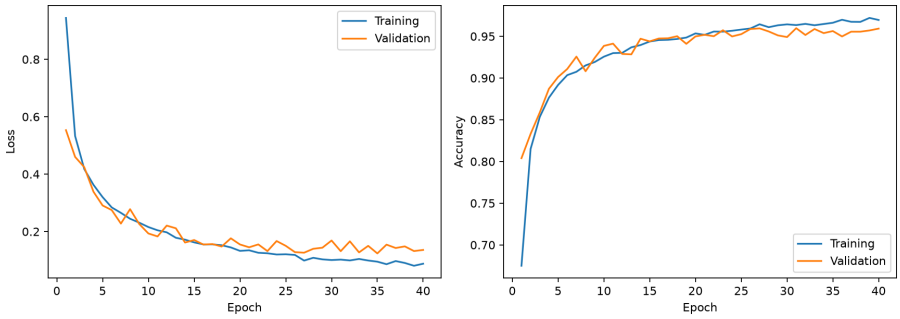


(b) SGD

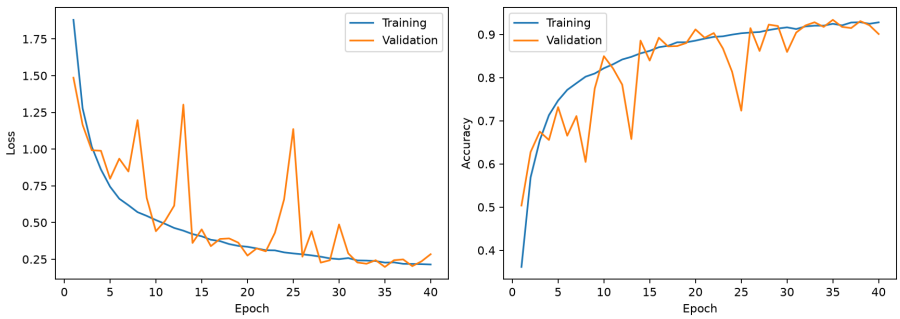


(c) SGD with momentum

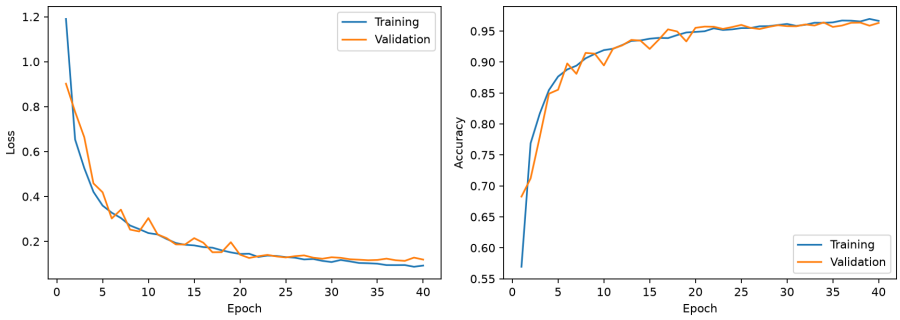
Figure 1: Model A: controlled training/validation loss and accuracy over 40 epochs.



(a) Adam

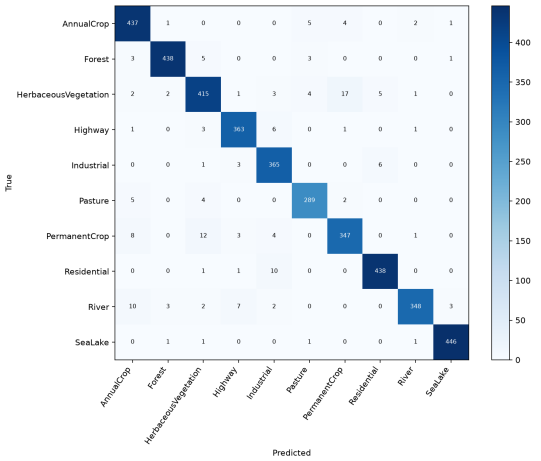


(b) SGD

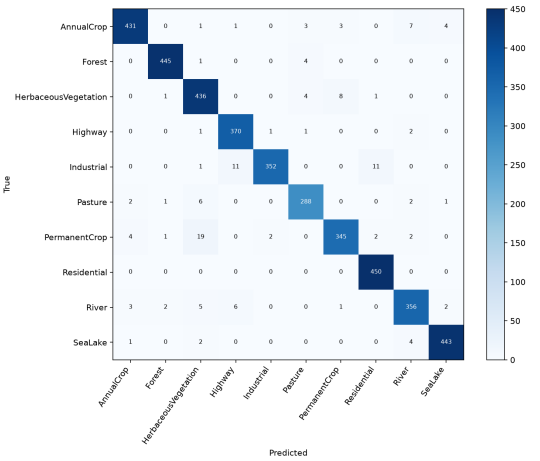


(c) SGD with momentum

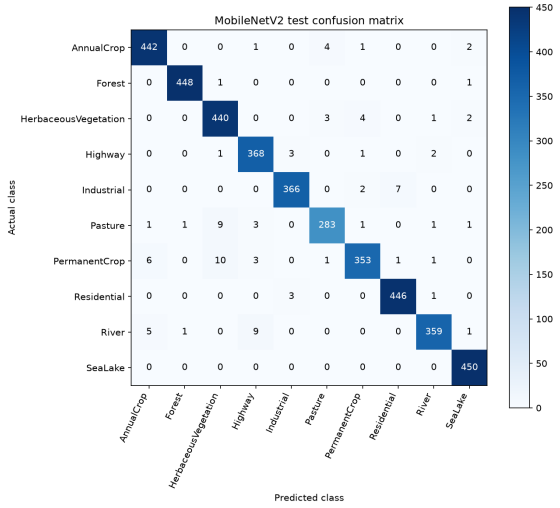
Figure 2: Model B: training/validation loss and accuracy for each optimizer over 40 epochs.



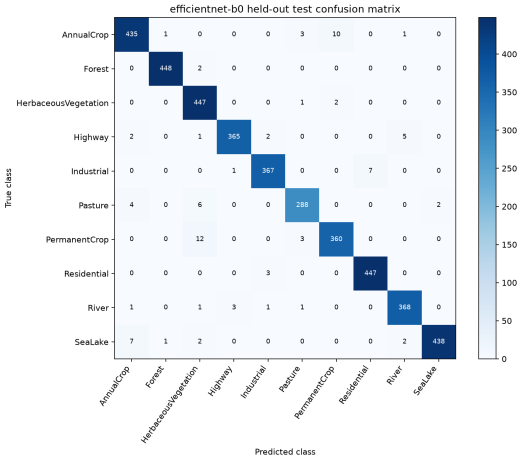
(a) Model A: momentum SGD



(b) Model B: Adam

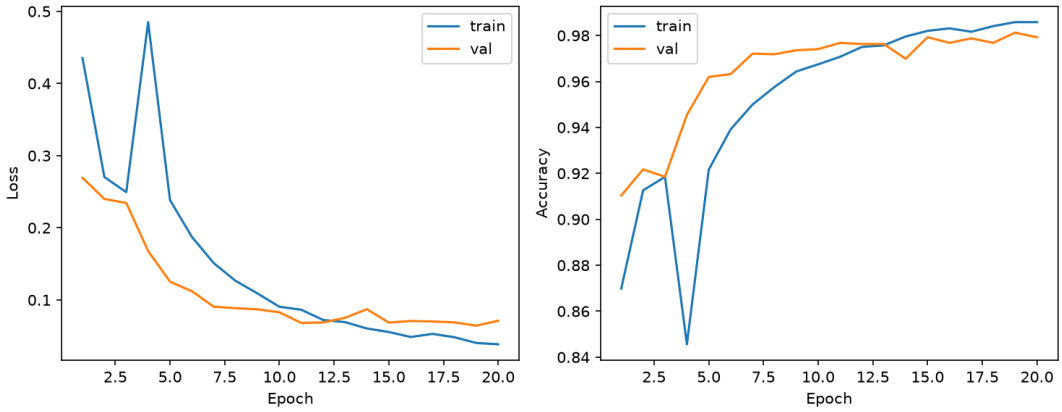


(c) MobileNetV2

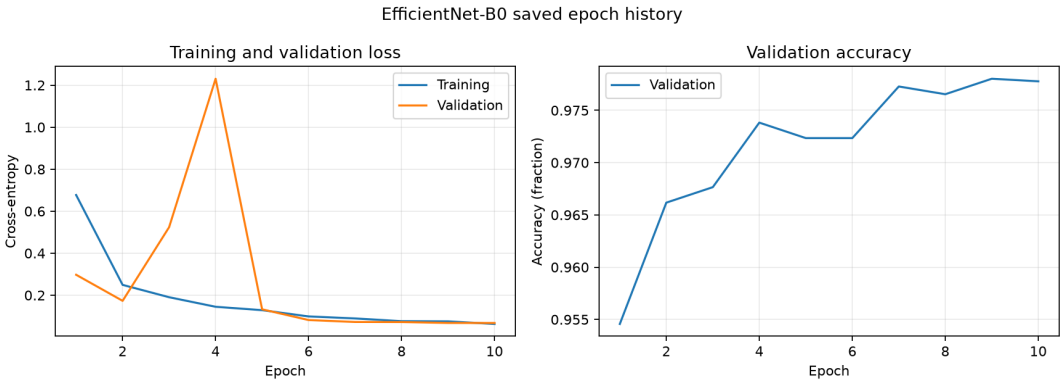


(d) EfficientNet-B0

Figure 3: Test confusion matrices for the displayed recipes. Rows are true classes and columns are predictions; each panel covers 4,050 images. Counts are not normalized and colour scales are panel-specific.



(a) MobileNetV2



(b) EfficientNet-B0

Figure 4: Pretrained-model training histories. EfficientNet-B0 did not log training accuracy, so its plot shows training/validation loss and validation accuracy only.

A Experiment records

The controlled study directory is:

`results/controlled_ab/controlled_ab_seed42_20261019_215110/`

Its `protocol.json` records shared settings, software/hardware, source hashes and dataset/split hashes. Each recipe has a 40-epoch history, a validation-best checkpoint, metrics and figures. The six histories and test matrices were checked for completeness and numerical consistency. Earlier unmatched runs remain historical records and are not the custom-model results reported here.

All report figures are copied into this report directory, so compilation does not require experiment folders. Checkpoints are preserved locally outside Git. MobileNetV2's original checkpoint remains unavailable locally; its saved metrics are reference artifacts, not independently reproduced predictions. EfficientNet-B0's older metadata lacks dataset and manifest hashes, limiting independent provenance verification.